Design Document: Hardware-Accelerated Block Game

Josh Bernheisel (jcb2301)
Wesley Maa (wm2505)
Mihir Joshi (mnj2122)

CSEE 4840 Embedded System Design

Spring 2026

Contents

1 Introduction **2**

2 System Block Diagram **2**
2.1 Block Descriptions . . . 2
2.2 Communication Pathways . . . 4

3 Algorithms **5**
3.1 Hidden-Face Culling (software) . . . 5
3.2 View-Frustum Culling (software) . . . 5
3.3 Vertex Transformation and Quad Setup (software) . . . 5
3.4 Quad Rasterization (hardware) . . . 6
3.5 VGA Scanout with Palette Lookup (hardware) . . . 7
3.6 Texture Lookup (hardware) . . . 7

4 Resource Budgets **7**
4.1 On-chip BRAM . . . 7
4.2 DDR3 (HPS side, 1 GB available) . . . 7
4.3 Rasterizer Throughput . . . 7
4.4 Command Buffer Bandwidth . . . 8

5 Hardware/Software Interface **8**
5.1 Control Register Map . . . 8
5.2 Quad Descriptor Format . . . 9
5.3 IOCTL Interface . . . 11
5.4 Per-Frame Software Sequence . . . 11

6 Milestones **11**

1 Introduction

We are building a hardware-accelerated 3D rendering engine on the DE1-SoC that produces a real-time, interactive block game on a VGA display. The player moves a first-person camera through a small procedurally generated world of textured cubes and can place or destroy blocks in real time.

The system partitions work between the ARM Cortex-A9 (running Linux) and custom FPGA logic. The ARM handles all control-heavy, branch-heavy computation: world representation, visibility determination, camera and matrix math, and per-quad rasterizer setup. The FPGA implements a fixed-function quadrilateral rasterizer with z-buffering, texture lookup, and VGA output generation, which is the massively parallel, arithmetic-regular part of the pipeline where custom hardware is most beneficial. A Linux kernel device driver mediates between the user-space game and the FPGA through the HPS-to-FPGA lightweight bridge.

Every face in a block world is a rectangle, so we rasterize **quadrilaterals** directly rather than tessellating into triangles. Under perspective projection, a world-space rectangle becomes a convex quadrilateral in screen space (assuming no vertex is behind the near plane, §3.3), which the four-edge-function test handles correctly. For v1 we emit one quad per exposed block face; greedy meshing to merge coplanar adjacent faces into larger rectangles is a natural optimization but is deferred past the first submitted milestone.

Target output: 320×240 internal resolution, 8-bit palette-indexed color, double-buffered, upscaled 2×2 to standard 640×480 VGA timing. This resolution fits comfortably in on-chip block RAM, eliminating the need for an SDRAM controller and giving the rasterizer deterministic single-cycle access to memory.

2 System Block Diagram

The system consists of ten major functional blocks split across software (user-space C on Linux) and hardware (SystemVerilog on the FPGA fabric), connected through a Linux kernel driver and the Avalon memory-mapped bus.

2.1 Block Descriptions

Software blocks (user space, running on ARM Cortex-A9 under Linux):

1. **Input Handler**: Reads USB keyboard and mouse events from `/dev/input/event*` using the Linux evdev API. Produces camera deltas (yaw, pitch, translation) and block place/destroy events each frame.

2. **Game Logic**: Maintains world state as 16×16×16 chunks of block IDs. Processes player actions and updates the world array in place.

3. **Visibility Stage**: Performs view-frustum culling on chunks (tests chunk AABB against six frustum planes), then hidden-face culling on blocks within visible chunks. Emits a list of visible quads.

4. **Geometry Stage**: Transforms each quad's four vertices through the combined model-view-projection matrix (Q16.16 fixed-point), performs perspective divide and viewport mapping. Computes edge-function coefficients (A_i, B_i, C_i), depth gradients, and UV gradients for each quad.

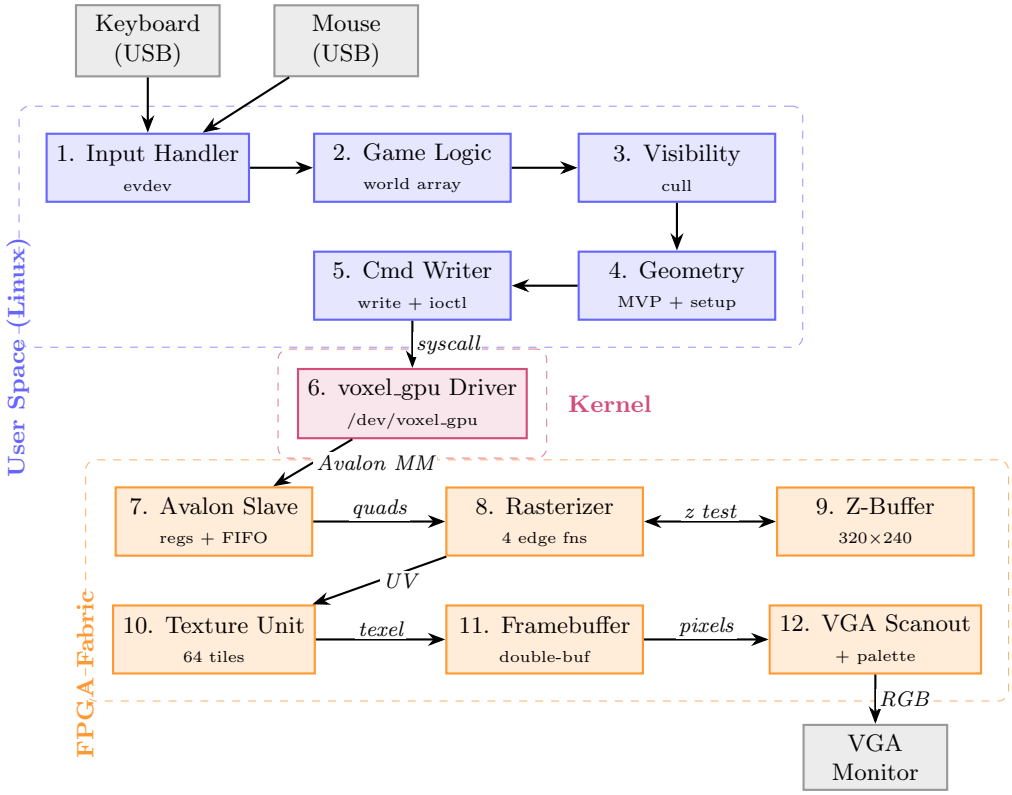

Figure 1: System block diagram. Data flows from USB input, through the Linux game on the ARM, across the Avalon bus via a kernel driver, into the FPGA rasterizer pipeline, and out to the VGA monitor. Dashed regions separate user-space software, kernel-space driver, and FPGA fabric.

5. **Command Buffer Writer**: Writes quad descriptors to the shared FIFO region via `write()` on `/dev/voxel_gpu`, then issues an ioctl to flip buffers.

Kernel-space block:

6. **voxel_gpu Device Driver**: Platform driver bound to `csee4840,voxel_gpu-1.0`. Maps the FPGA's lightweight bridge registers, exposes `/dev/voxel_gpu`, and implements ioctls for palette upload, frame submission, and status query.

Hardware blocks (FPGA fabric):

7. **Avalon MM Slave**: Receives 32-bit writes from the ARM to control registers and to the memory-mapped command FIFO window. Exposes status registers back.
8. **Quad Rasterizer**: Pops quad descriptors from the command FIFO, iterates each quad's bounding box, evaluates four edge functions per pixel, interpolates depth and UV coordinates incrementally.
9. **Z-Buffer Unit**: 16-bit depth buffer at 320×240 in BRAM (150 KB). Compares interpolated depth against stored depth; gates framebuffer writes.
10. **Texture Unit**: Stores up to 64 16×16 palette-indexed texture tiles in BRAM. Nearest-neighbor UV lookup.
11. **Framebuffer**: Double-buffered 320×240 8bpp framebuffers in BRAM.
12. **VGA Scanout + Palette**: VGA timing controller reads the front buffer, looks up RGB888 values through a 256-entry palette BRAM, and drives the on-board ADV7123 DAC at 640×480@60Hz with 2×2 pixel doubling.

2.2 Communication Pathways

- **USB → Linux → Input Handler**: HID events polled per frame via evdev.
- **Game Logic ↔ Visibility ↔ Geometry ↔ Command Writer**: In-process function calls, all user-space C; data structures are 3D `uint8_t` block arrays and `quad_desc_t` structs.
- **User space → Driver**: `write()` for bulk command stream data, `ioctl()` for control (palette, flip, status). Driver uses `copy_from_user` then `iowrite32` across the bridge.
- **Driver → Avalon MM Slave**: 32-bit writes over the HPS-to-FPGA lightweight bridge at base `0xFF20_0000`.
- **Avalon Slave → Rasterizer**: 8 KB command FIFO in BRAM; ARM pushes descriptors, rasterizer pops them.
- **Rasterizer → Z-buffer / Framebuffer**: Direct BRAM ports at one access per pixel clock.
- **Framebuffer → Scanout → Palette → DAC**: Three-stage pipeline at the 25.175 MHz VGA pixel clock.

3 Algorithms

3.1 Hidden-Face Culling (software)

For each solid block in a visible chunk, check all six axis-aligned neighbors. Emit a quad for a face only if the neighbor on that side is air (or outside the chunk boundary with the adjacent chunk's neighbor block air). This eliminates interior geometry.

```
for each chunk C in visible_chunks:
  for each block B at (x,y,z) in C:
    if B is air: continue
    for each direction d in {+x, -x, +y, -y, +z, -z}:
      N = block_at(x,y,z + offset(d))
      if N is air:
        emit_quad(B, d)
```

3.2 View-Frustum Culling (software)

Before per-block work, reject entire chunks whose axis-aligned bounding boxes fall outside the camera frustum. Each chunk's 8 corners are tested against the 6 frustum planes derived from the view-projection matrix. If all 8 corners lie on the outside half-space of any single plane, the chunk is culled. Some chunks partially outside the frustum will pass, but no visible chunk is ever rejected.

3.3 Vertex Transformation and Quad Setup (software)

For each emitted quad (four vertices in world space):

1. Multiply each vertex by the combined model-view-projection matrix (4×4, Q16.16 fixed-point).

2. **Near-plane handling.** If any of the four vertices has $w \leq w_{\text{near}}$, the quad straddles or lies behind the near plane; perspective divide would produce invalid (negative or enormous) screen coordinates and the projected shape may be non-convex, breaking the edge-function test. For v1 we *reject* such quads outright (cheap, one comparison per vertex). The visual artifact is that a face partially behind the camera pops out of view one frame early; acceptable given a small near-plane distance. A future revision can replace rejection with Sutherland–Hodgman clipping against the near plane, which splits the quad into a convex polygon of up to five vertices — more work for the geometry stage but no change to the rasterizer contract.

3. Perspective divide: divide x, y, z by w.

4. Viewport map: $\text{screen}_x = (x + 1) \cdot 160$, $\text{screen}_y = (1 - y) \cdot 120$.

5. Compute four edge coefficients. For an edge from (x_0, y_0) to (x_1, y_1):

$$A = y_0 - y_1, \quad B = x_1 - x_0, \quad C = -(A \cdot x_0 + B \cdot y_0)$$

   The edge function $E(x, y) = Ax + By + C$ is non-negative on the interior side for CCW winding.

6. Compute depth plane coefficients $z(x, y) = z_0 + \frac{dz}{dx}x + \frac{dz}{dy}y$ from the three screen-space vertices with their depths.

7. Compute UV gradients for textured quads.
8. Compute the screen-space bounding box, clamped to $[0, 319] \times [0, 239]$.

The output is one `quad_desc_t` per quad, pushed to the command FIFO.

3.4 Quad Rasterization (hardware)

For each quad descriptor popped from the FIFO:

```
// Initialize at (x_min, y_min)
E0 = A0*x_min + B0*y_min + C0 (similarly E1, E2, E3)
z  = z0 + dz_dx*x_min + dz_dy*y_min
u, v = similarly

for y = y_min to y_max:
  save row-start values of E0..E3, z, u, v
  for x = x_min to x_max:
    if (E0 >= 0) and (E1 >= 0) and (E2 >= 0) and (E3 >= 0):
      z_cur = z_buffer[x, y]
      if z < z_cur:
        color_idx = texture_fetch(u, v, texture_id)
        framebuffer_back[x, y] = color_idx
        z_buffer[x, y] = z
    E0 += A0; E1 += A1; E2 += A2; E3 += A3
    z  += dz_dx; u += du_dx; v += dv_dx
  restore row-start; step: E_i += B_i; z += dz_dy; etc.
```

The crucial property exploited in hardware: edge functions, depth, and UV all advance by a *constant* when stepping one pixel in x or y. Per-pixel work reduces to four adds and four sign checks in the inner loop. This maps directly to a shallow pipeline.

BRAM port arrangement. Each rasterized pixel touches multiple on-chip memories, but careful port assignment keeps steady-state throughput at one pixel per cycle.

- **Z-buffer:** read-modify-write per pixel. Implemented on a true dual-port M10K with the read port issuing the depth fetch on cycle n and the write port committing the update on cycle $n + 1$ — a two-stage pipeline sustains one access per cycle. Alternatively, a single-port BRAM clocked at $2\times$ the rasterizer rate would serve both accesses; we prefer the dual-port approach for simpler timing closure.
- **Framebuffer:** one write per passing pixel on the rasterizer-facing port. The opposite port belongs to the scanout pipeline. Because the design is double-buffered (the rasterizer writes the *back* buffer while scanout reads the *front*), there is no port contention between the two domains — they target different BRAMs until `VGPU_FLIP`.
- **Texture BRAM:** one read per textured pixel. Single-port sufficient; rasterizer stalls one cycle if a simultaneous access is needed (not triggered in the current design).
- **Z-buffer clear (`CLR`).** Clearing $320\times240 = 76{,}800$ entries is a linear sweep over the Z-buffer, one write per FPGA cycle. At 50 MHz that takes $\sim$1.5 ms, short relative to the 16.6 ms frame budget; the rasterizer stays idle via `BSY`/`CLR` handshake until the sweep completes.

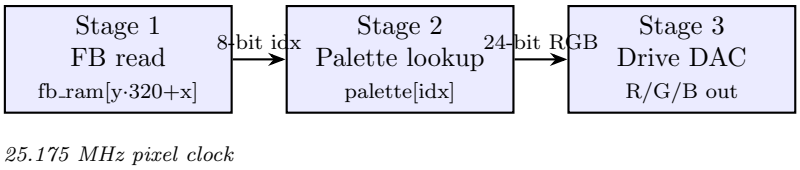

Figure 2: VGA scanout pipeline. Each stage is one cycle at the 25.175 MHz pixel clock; after priming, one pixel is driven to the DAC per clock.

3.5 VGA Scanout with Palette Lookup (hardware)

A three-stage pipeline at the 25.175 MHz pixel clock:

2×2 pixel doubling. The framebuffer is 320×240 but VGA timing drives 640×480, so the scanout address is computed as

$$\text{fb_addr} = \text{hcount}[9{:}1] + 320 \cdot \text{vcount}[8{:}1],$$

i.e., the low bit of each counter is dropped. The same address is therefore presented to the framebuffer port for two consecutive pixel clocks horizontally and on two consecutive lines vertically, so every 320×240 logical pixel appears as a 2×2 block at 640×480. A naive implementation re-reads the framebuffer every cycle despite the address not changing; a more efficient variant latches the color and only issues BRAM reads on even `hcount`, but both produce identical output. The scanout side sees the framebuffer through its dedicated BRAM port, independent of the rasterizer side (see Z-buffer / framebuffer discussion in §3.4).

3.6 Texture Lookup (hardware)

Textures are 16×16 palette-indexed tiles stored in BRAM. The rasterizer carries u and v as Q16.16 fixed-point (matching the descriptor format); only the low four integer bits select the texel. Nearest-neighbor lookup:

$$\text{texel} = \text{tex_ram}[\text{texture_id}][\lfloor u \rfloor \bmod 16][\lfloor v \rfloor \bmod 16]$$

The output is an 8-bit palette index, written directly to the framebuffer.

4 Resource Budgets

4.1 On-chip BRAM

The Cyclone V on the DE1-SoC has approximately 4 Mbit (≈512 KB) of on-chip M10K block RAM.

4.2 DDR3 (HPS side, 1 GB available)

4.3 Rasterizer Throughput

At 50 MHz FPGA clock with a single-pixel-per-cycle rasterizer, theoretical peak is 50 Mpix/s. A fully-covered 320×240 frame is 76,800 pixels = 1.5 ms of rasterizer time. At 60 fps (16.6 ms budget), we can afford ∼10× overdraw. Typical first-person views of voxel terrain have depth complexity in the 2–4× range after back-face and hidden-face culling, well within budget even without occlusion culling between surviving faces.

Resource	Size	Notes
Framebuffer ×2 (320×240, 8bpp)	153.6 KB	Double-buffered
Z-buffer (320×240, 16-bit)	153.6 KB	Cleared per frame
Palette (256 × RGB888)	0.75 KB	Uploaded at startup
Texture tiles (64 × 16×16 × 8bpp)	16 KB	Expandable
Quad command FIFO	8 KB	∼128 quads in flight
Total BRAM	**∼332 KB**	**∼65% of available**

Table 1: On-chip block RAM budget. Extra headroom for expansion if needed.

Resource	Size
World data (up to 32 chunks loaded, 4 KB each)	∼128 KB
Per-frame visible quad list	<64 KB
Linux kernel + user program + assets	<64 MB

Table 2: DDR3 usage on the HPS side; effectively unbounded relative to our needs.

4.4 Command Buffer Bandwidth

Flat-shaded descriptors are 64 bytes; textured descriptors are 128 bytes (see §5.2). Assuming the worst case — every visible quad textured — at 60 fps with ∼500 visible quads per frame (generous post-cull estimate): $500 \cdot 128 \cdot 60 \approx 3.84$ MB/s over the lightweight bridge, which sustains tens of MB/s. Not a bottleneck.

5 Hardware/Software Interface

The ARM communicates with the rasterizer through two mechanisms: a small set of **control registers** at fixed Avalon MM offsets for commands and status, and a larger **memory-mapped FIFO window** for streaming quad descriptor data.

5.1 Control Register Map

Base address: `0xFF20_0000` (assigned by Platform Designer; actual value determined from the device tree). All registers are 32 bits. Addresses are byte offsets from base.

Offset 0x00: CONTROL (R/W)

[31:3]	[2]	[1]	[0]
reserved	CLR	FLP	EN

Bit 0	EN	Rasterizer enable. 0 = idle, safe to reconfigure; 1 = process FIFO.
Bit 1	FLP	Buffer flip request. Write 1 to swap front/back on next vsync. Auto-clears.
Bit 2	CLR	Z-buffer clear. Write 1 to clear; auto-clears when done.
Bits 31:3	—	Reserved, write 0.

For v1, software drives frame pacing by *polling* `STATUS.FEM` (FIFO empty) and `STATUS.VSY` (vblank) rather than taking a hardware interrupt. At 60 fps the polling cost is negligible (tens of reads per 16 ms frame).

Offset 0x04: STATUS (R only)

[31:16]	[15:4]	[3]	[2]	[1]	[0]
reserved	FIFO_WORDS	VSY	FEM	FFL	BSY

Bit 0	BSY	Rasterizer busy (actively processing a quad).
Bit 1	FFL	Command FIFO full; writes will be dropped.
Bit 2	FEM	Command FIFO empty.
Bit 3	VSY	Currently in vertical blanking interval (safe to flip).
Bits 15:4	FIFO_WORDS	Number of 32-bit words currently queued in the command FIFO, 0–2048.
Bits 31:16	—	Reserved.

The FIFO reports its fill level in *words* rather than descriptors because the hardware cannot know descriptor boundaries at enqueue time — a flat-shaded quad is 16 words (64 bytes) and a textured quad is 32 words (128 bytes). Software divides by 16 (or 32) as appropriate when deciding whether a push will fit.

Offset 0x08: FRAME_COUNT (R only) — 32-bit counter incremented once per completed frame. Used by software for FPS measurement.

Offset 0x0C: PALETTE_ADDR (W only) — palette entry index (0–255) to program next. Writing arms the palette write port.

Offset 0x10: PALETTE_DATA (W only)

[31:24]	[23:16]	[15:8]	[7:0]
reserved	RED	GREEN	BLUE

Writing `PALETTE_DATA` commits the RGB888 value to `palette[PALETTE_ADDR]` and auto-increments `PALETTE_ADDR`. At startup, software writes `PALETTE_ADDR` once, then `PALETTE_DATA` 256 times to upload the full palette.

Offsets 0x1000–0x2FFF: FIFO_WINDOW (W only) — 8 KB memory-mapped window into the command FIFO. Writes are enqueued as quad descriptor data. Software writes 16 consecutive 32-bit words to push one 64-byte flat-shaded descriptor, or 32 consecutive words (128 bytes) for a textured descriptor.

5.2 Quad Descriptor Format

Each descriptor is 64 bytes, written as 16 consecutive 32-bit little-endian words through `FIFO_WINDOW`. The rasterizer pops one descriptor at a time in exactly this layout.

Depth format rationale. Viewport-mapped depth lands in [0, 1] nominally, but the interpolated value $z_0 + \mathrm{d}z_{\mathrm{d}x} \cdot \Delta x + \mathrm{d}z_{\mathrm{d}y} \cdot \Delta y$ at the far corner of a large bounding box can slightly exceed 1 due to fixed-point rounding and edge-function inclusivity. Q1.15 for `z0` (range $[0, 2)$) absorbs this margin while keeping the same 16-bit width as the Z-buffer. Per-pixel depth gradients in a 320×240 viewport with near/far clip at 0.1/100 never exceed a few parts per thousand, so signed Q1.15 gives $\sim 3 \times 10^{-5}$ precision — a better fit than Q4.12 given the actual dynamic range.

Flags byte (offset 63):

Bit 0	TEX	1 = textured (second block follows); 0 = flat color
Bit 1	ZTEST	1 = enable z-test and z-write; 0 = always pass
Bits 7:2	—	Reserved

Offset	Size	Field	Format	Meaning
0	2	`x_min`	int16	Bounding box left
2	2	`y_min`	int16	Bounding box top
4	2	`x_max`	int16	Bounding box right
6	2	`y_max`	int16	Bounding box bottom
8	4	`edge0_A`	int32 Q16.16	Edge 0 coefficient A
12	4	`edge0_B`	int32 Q16.16	Edge 0 coefficient B
16	4	`edge0_C`	int32 Q16.16	Edge 0 coefficient C
20	4	`edge1_A`	int32 Q16.16	Edge 1 A
24	4	`edge1_B`	int32 Q16.16	Edge 1 B
28	4	`edge1_C`	int32 Q16.16	Edge 1 C
32	4	`edge2_A`	int32 Q16.16	Edge 2 A
36	4	`edge2_B`	int32 Q16.16	Edge 2 B
40	4	`edge2_C`	int32 Q16.16	Edge 2 C
44	4	`edge3_A`	int32 Q16.16	Edge 3 A
48	4	`edge3_B`	int32 Q16.16	Edge 3 B
52	4	`edge3_C`	int32 Q16.16	Edge 3 C
56	2	`z0`	uint16 Q1.15	Depth at $(x_{\min}, y_{\min})$; range [0, 2)
58	2	`dz_dx`	int16 Q1.15	Depth gradient, x; range $[-1, 1)$
60	2	`dz_dy`	int16 Q1.15	Depth gradient, y; range $[-1, 1)$
62	1	`texture_id`	uint8	Texture index (0–63)
63	1	`flags`	uint8	See below

Table 3: Quad descriptor (64 bytes). Textured quads append a second 64-byte block carrying UV origin and gradients; flat-shaded quads omit the second block.

Second block (textured quads only, 64 bytes): When the `TEX` flag is set, 16 additional 32-bit words follow the first block in `FIFO_WINDOW`, carrying UV origin and gradients.

Offset	Size	Field	Format	Meaning
0	4	`u0`	int32 Q16.16	u at $(x_{\min}, y_{\min})$
4	4	`v0`	int32 Q16.16	v at $(x_{\min}, y_{\min})$
8	4	`du_dx`	int32 Q16.16	u gradient along x
12	4	`dv_dx`	int32 Q16.16	v gradient along x
16	4	`du_dy`	int32 Q16.16	u gradient along y
20	4	`dv_dy`	int32 Q16.16	v gradient along y
24	40	`reserved`	—	Write 0; reserved for future per-quad material data

Table 4: Textured-quad UV block (64 bytes). Carries affine UV interpolation state consumed by the texture unit. The reserved tail keeps the block 64 bytes so all descriptors are a fixed multiple of 64.

Per-pixel UV is interpolated incrementally using the same adder-per-step scheme as the edge functions: $u \mathrel{+}= du_dx$ per x step, $u \mathrel{+}= du_dy$ per row (and similarly for v). The rasterizer fetches a texel via nearest-neighbor lookup into the tile indexed by `texture_id`.

Texture upload. For v1, the 64 texture tiles will be baked into BRAM at synthesis using `$readmemh` against a hex file produced by an offline asset pipeline; no runtime texture upload path exists. A second write window analogous to the palette port is a straightforward extension if dynamic texture loading becomes necessary.

5.3 IOCTL Interface

The device driver exposes `/dev/voxel_gpu` with the following ioctls:

ioctl	Argument	Function
`VGPU_SET_PALETTE`	`struct {uint8_t idx,r,g,b;}`	Upload one palette entry
`VGPU_FLIP`	none	Flip on next vsync (blocking)
`VGPU_CLEAR_Z`	none	Clear z-buffer (blocking)
`VGPU_GET_STATUS`	`uint32_t*` (out)	Read STATUS register
`VGPU_GET_FRAME_COUNT`	`uint32_t*` (out)	Read FRAME_COUNT

Quad descriptor writes happen through ordinary `write()` calls on the device, which the driver forwards to `FIFO_WINDOW` in bursts using `iowrite32`.

5.4 Per-Frame Software Sequence

```
for each frame:
  1. poll input, update game state
  2. ioctl(fd, VGPU_CLEAR_Z)
  3. cull chunks, cull faces, transform quads -> quad_desc[]
  4. write(fd, quad_desc, n * 64)
  5. ioctl(fd, VGPU_FLIP) // blocks until rasterizer drains + vsync
```

6 Milestones

The project is structured so that each phase produces a runnable, end-to-end system — partial credit at any checkpoint. The team is split into three roles (bus/command path, rasterizer core, display path) so that milestones 1–3 can progress in parallel once the platform scaffolding is in place.

1. **VGA + framebuffer baseline.** FPGA VGA timing controller displays a test pattern from a 320×240 BRAM framebuffer, 2×2 upscaled to 640×480. ARM pokes individual pixels via the bridge to confirm the full write path. Extends lab 3's `vga_ball` with a real frame buffer plus the palette BRAM and scanout pipeline.

2. **Linux device driver.** Implement `voxel_gpu` platform driver bound to `csee4840,voxel_gpu-1.0`. Expose `/dev/voxel_gpu` with `write()`, the five ioctls from §5.2, and `VGPU_SET_PALETTE`. User-space test program uploads a palette and streams pixels.

3. **Software renderer prototype.** Full user-space C pipeline: world representation, frustum and hidden-face culling, MVP transform, quad setup, and software rasterization writing pixels to the framebuffer through the driver. Proves the geometry and culling code correct independent of FPGA hardware.

4. **FPGA quad rasterizer.** SystemVerilog implementation of `quad_fetch`, `rasterizer`, `edge_eval`, and `interp_unit`. Verified in simulation (Verilator/ModelSim) against hardcoded quad descriptors before integration. Add the Z-buffer and verify z-rejection on overlapping quads.

5. **Integration.** Wire the user-space pipeline to the FPGA rasterizer via the driver. Render a small static scene of flat-shaded cubes using hardware acceleration at ≥30 fps.

6. **Camera and interaction.** First-person camera movement with USB keyboard and mouse (evdev). Block placement and destruction via mouse buttons, with incremental chunk remeshing.

7. **Texturing.** Populate texture BRAM via `$readmemh`, enable the `TEX` flag path in the rasterizer, and add per-pixel UV interpolation and nearest-neighbor lookup. Final deliverable: a textured block world the player can walk through and modify at 60 fps.

Stretch goals (if time permits after milestone 7): procedural world generation, simple lighting (per-face flat shading from a fixed sun direction), a skybox quad, and IRQ-driven frame pacing in place of polled `VGPU_FLIP`.